

双重智能体困境

- 时间限制：12 分钟。
- 存储空间：5 GB
- 环境：一块 GPU (≈ 16 GB VRAM)，无互联网
- 解答大小：`solution.ipynb` ≤ 1 MB
- 基线分数：0

在阿斯塔纳的国家人工智能中心，两个计算机模型——Model R（一个 ResNet-18）和 Model V（一个 ViT-Tiny）——正在分析照片。目前，这两个模型都表现完美，准确率达到 100%，并且在每一张图像 (every single image) 上的判断都一致。为了检验它们的智能“大脑”究竟有多大差异，首席科学家向你提出了一项挑战：对每张照片的像素进行微小且几乎不可见的改动，使 Model R 和 Model V 的判断完全不一致 (**completely disagree**)。



1. 任务

两个预训练图像分类器查看同一张图像。在本任务提供的图像上，这两个分类器均达到 100% 的准确率。

- **Model R:** `torchvision.models.resnet18`（一个 CNN，ResNet18）。
- **Model V:** `timm` 的 `vit_tiny_patch16_224`（一个 Transformer，ViT-Tiny）。

你的任务是为每张图像创建一个微小改动（“扰动 **perturbation**”），使两个模型的判断不一致。对于每张图像，你必须创建**两种不同的扰动**：

- **类型 A**：添加该扰动后，Model R 仍能正确分类该图像，但 Model V 会错误分类。
- **类型 B**：添加该扰动后，Model V 仍能正确分类该图像，但 Model R 会错误分类。

每个扰动都必须足够小，以至于难以察觉。扰动越小，得分越高（见第 5 节）。扰动直接在像素层面应用于原始图像。

2. 公开数据

任务提供了一组图像，分为两个数据划分——`train`（100 张图像）和 `test_public`（100 张图像）——每个划分都包含**分辨率各异 (varying resolution)** 的图像。所有图像均来自 ImageNet-1K 的 1000 个类别，并且 Model R 和 Model V 在这两个划分上都达到 100% 的准确率。

提供以下文件：

```
train/images/*.png          # 100 images in PNG format
train/labels.json           # maps each image index to its correct class
test_public/images/*.png    # 100 images in PNG format
test_public/labels.json     # maps each image index to its correct class
```

评分时，你的 `dataset/test_public/` 文件夹会被透明地替换为两个隐藏图像集（`test_leaderboard_a` 和 `test_leaderboard_b`），用于正式评分。每个图像集均包含 **100 张图像**，图像格式为 PNG，并附有一个标签文件。

注意：对于本任务，测试数据集中的标签可以访问。(Note: For this task, the labels in test datasets are accessible.)

3. 输出格式

对于每张图像，你必须生成两个文件：

```
{index}_a.pt    # Type A perturbation
{index}_b.pt    # Type B perturbation
```

- `{index}`（0、1、2、……）与数据集中图像的名称一致 (**matches the image's name in the datasets**)。
- 每个文件都是一个使用 `torch.save` 保存的张量 (tensor)。其形状必须为 $3 \times H \times W$ ，其中 H 和 W 与该图像的**原始分辨率一致（不是 224×224 ）**。
- 代码应只生成一个 ZIP 文件，即 `submission.zip`。将所有 `.pt` 文件放在 ZIP 压缩包的顶层，不要包含外层文件夹或子目录 (with no enclosing folder or subdirectories)。

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

如果输出格式存在任何问题，notebook 将向你发出警告。

4. 约束

- **模型：**你必须使用 `torchvision.models.resnet18(pretrained=True)` 和 `timm.create_model('vit_tiny_patch16_224', pretrained=True)`。不允许使用任何其他预训练模型。
- **变换流水线（评测时强制执行）(Transform pipeline (enforced at evaluation))：**

`Resize(256) → CenterCrop(224) → Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])`。See Section 3 of `baseline.ipynb`，详见相关内容。

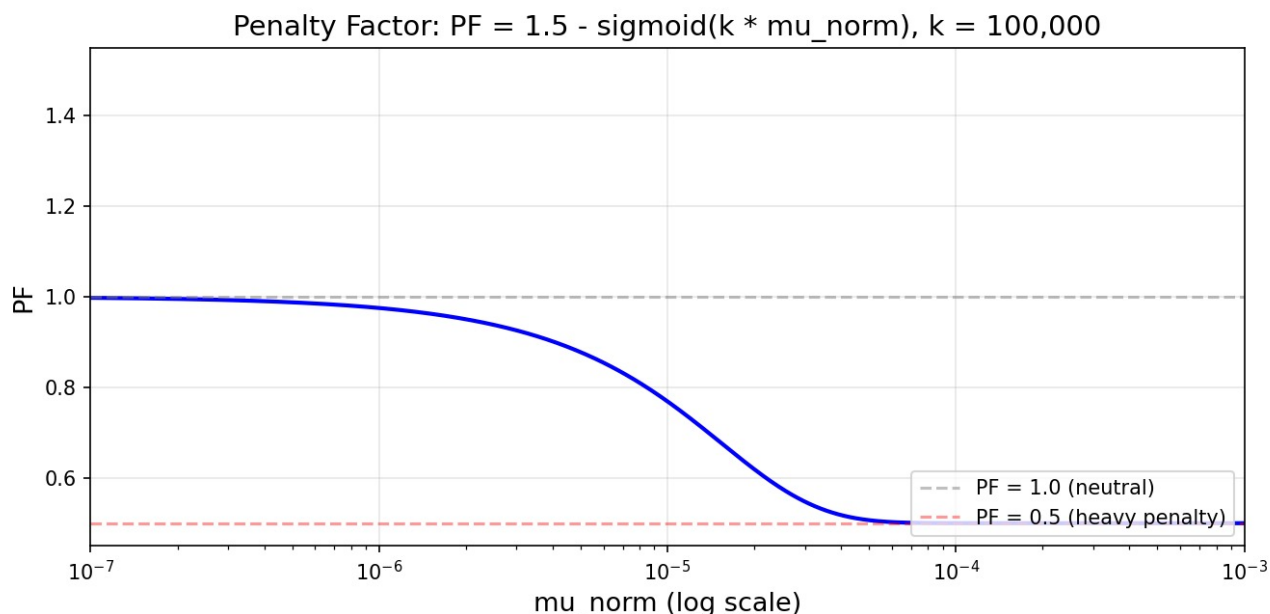
- **扰动分辨率：**必须与原始图像的原始分辨率一致（**不是** 224×224 ）。张量会在变换流水线执行之前加到原始图像中(The tensor is **added** to the raw image **before** the transform pipeline.)。
- **输出格式：**仅允许 `.pt` 文件——不允许 PNG/JPG。Tensors are added to the image (**normalized to** `[0, 1]`) and then values are clipped to `[0, 1]` before preprocessing.
- **文件命名：**文件必须平铺列出，严格采用 `{index}_a.pt / {index}_b.pt` 格式。zip 内不得包含子目录。
- **库：**`torch`、`torchvision`、`timm`。

5. 评分

最终分数按以下方式计算。设 M 为该划分中的图像数量， $Score_A$ 为成功的类型 A 扰动数量， $Score_B$ 为成功的类型 B 扰动数量：

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF (惩罚函数) 是一个用于惩罚具有较高范数的扰动 (**penalise perturbations with a high norm**)，并在性能接近上限时非常敏感的函数 (**very sensitive near the ceiling of performance**)。它被限制在 **0.5 到 1** 的范围内。完整实现可见 `solution.ipynb` 的第 8 节 (**Section 8**)。



图：惩罚函数 (PF) 的曲线。

6. 检查提交内容

notebook 中包含检查项，如果存在格式问题，它们会向你发出警告；这些检查项位于 `solution.ipynb` notebook 的第 7 节 (**Section 7**)。

7. 本地测试

`solution.ipynb` 包含一个完整且可运行的示例。它会加载公开数据、两个模型和官方评分器，并写出一个提交用 ZIP 文件。开始前请先阅读它。

8. 如何提交

- 将你的更改保存到 `solution.ipynb`。
- 打开 JupyterLab 左侧边栏中的 Git 选项卡。
- 将 `solution.ipynb` **暂存 (Stage)** (点击其旁边的 + 图标)。
- 输入提交消息，然后点击 **Commit**。
- 点击带向上箭头的云图标进行推送 (push)。
- 返回本竞赛页面，然后点击 **Submit**。

只提交一个文件，文件名为 `solution.ipynb`。